ACL 2025 VIENNA

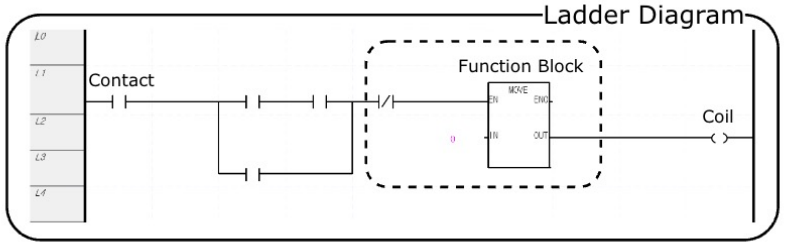
Retrieval-Augmented Fine-Tuning With Preference Optimization For Visual Program Generation

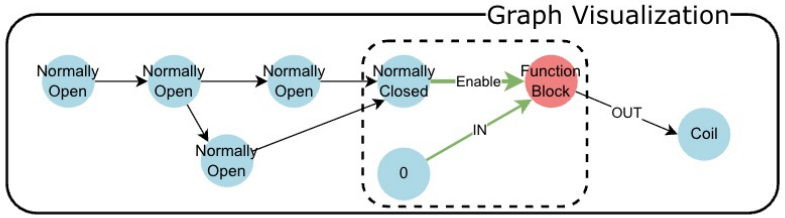
Deokhyung Kang[1]* **Jeonghun Cho[1]*** **Yejin Jeon[1]** **Sunbin Jang[2]**
Minsub Lee[2] **Jawoon Cho[3]** **Gary Geunbae Lee[1]**
[1]Pohang University of Science and Technology
[2]Hyundai Mobis [3]T&I Company

POSTECH HYUNDAI MOBIS T&I COMPANY TRUST AND INNOVATION FOR YOU

Problem Statement

Ladder diagram






```
<Rung>
  <Element Type = "NomarllyOpen" Coordinate = "1"></Element>
  <Element Type = "Line" Coordinate = "3"></Element>
  <Element Type = "FunctionBlock" Coordinate = "9"</Element>
   . . .
</Rung>
```

XML

```
{"0": {"attributes": . . .,
      "edge": {(1, "Enable")}},
"1": {"attributes": . . .,
      "edge": {(2, "same")}},
. . .}
```

JSON

```
import networkx as nx
G = nx.Digraph()
G.add_node(0, . . .)
G.add_node(1, . . .)
G.add_edge(0, 1, type=Ena..)
. . .
```

Metaprogram

- Ladder Diagram (LD) is a visual language for controlling factory machines via Programming Logic Controllers (PLCs).
- Its circuit-like design makes it easy for engineers to build control logic, so it's widely used in PLCs.
- Since LLMs can't generate visuals, we convert LD into a graph and represent it in XML, JSON, and Metaprogram formats.

Problem Statement

Datasets

Type	Content
Prompt	**Program Description:** 셀 이재기 서보 조그 고속 하한 값 인터락 프로그램을 만들어줘. (Create a cell transfer servo jog high-speed interlock program.) **Detailed Description:** 조그 고속 속도 설정값이 20 이하일 때 조그 고속속도를 20으로 설정해줘. (When the jog high-speed setting value is 20 or less, set the jog high-speed to 20.)

XML (Anonymized)	`<Rung>`
	`  <Element Attributes..., Coordinate="X"></Element>`
	`  <Element Attributes..., Coordinate="Y"></Element>`
	`  <Element Attributes..., Coordinate="Z"></Element>`
	`</Rung>`

Problem Statement

Datasets

Type	Content
Prompt	**Program Description:** 셀 이재기 서보 조그 고속 하한 값 인터락 프로그램을 만들어줘. (Create a cell transfer servo jog high-speed interlock program.) **Detailed Description:** 조그 고속 속도 설정값이 20 이하일 때 조그 고속속도를 20으로 설정해줘. (When the jog high-speed setting value is 20 or less, set the jog high-speed to 20.)

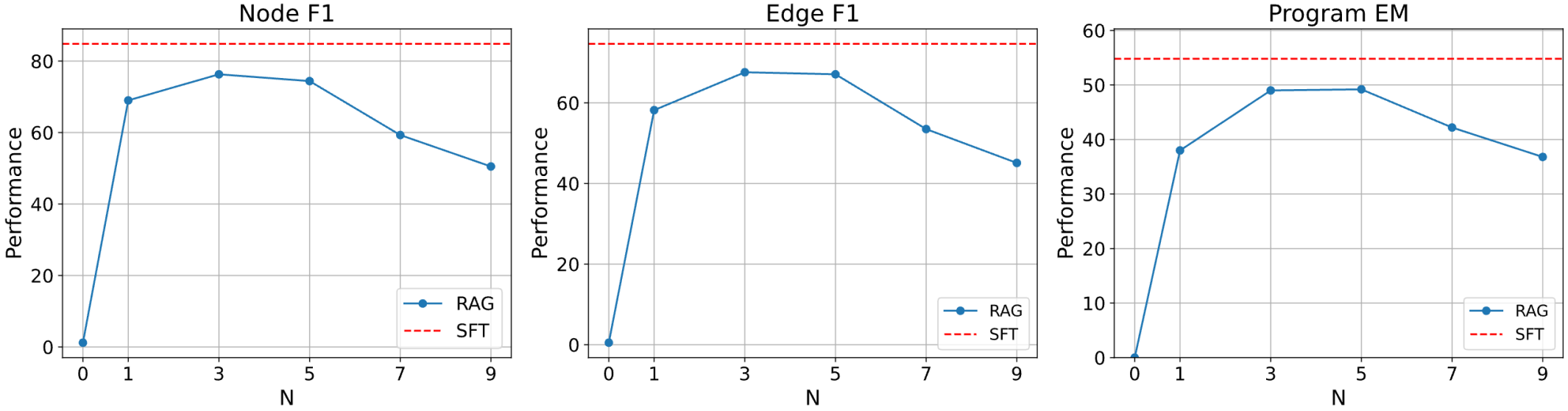
XML (Anonymized)	`<Rung>`
	`  <Element Attributes..., Coordinate="X"></Element>`
	`  <Element Attributes..., Coordinate="Y"></Element>`
	`  <Element Attributes..., Coordinate="Z"></Element>`
	`</Rung>`

Process Type	Count	Ratio (%)
Cell Distribution	2,893	20.5
Cell Processing	3,093	21.9
Cell Vision Inspection	726	5.1
Cell Stacking	3,706	26.2
Pack End-plate Supply	2,808	19.9
Other Processes	898	6.4

Challenges

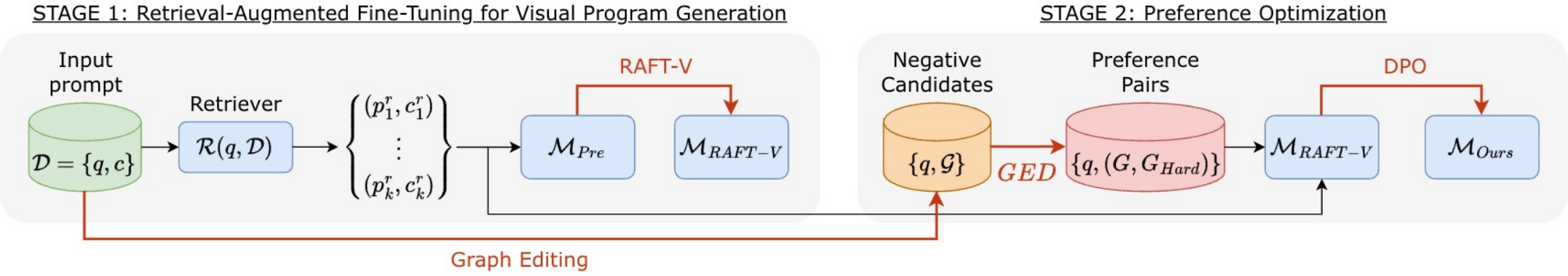
RAG vs. SFT

- Prompting-based methods struggle with LD generation.



Proposed Method

Two-stage training-based approach



- Input: User prompt $\mathbf{q}$ + Retrieved examples $[(\mathrm{p}_1^{\mathrm{r}}, \mathrm{c}_1^{\mathrm{r}}), .. (\mathrm{p}_{\mathrm{k}}^{\mathrm{r}}, \mathrm{c}_{\mathrm{k}}^{\mathrm{r}})]$ -> Output: Generated LD $\mathbf{c}$

- Stage 1: Retrieval-Augmented Fine-Tuning (RAFT-V) : We retrieve similar prompt-code pairs from the dataset and fine-tune the model with them, both during training and inference.

- Stage 2: Preference Optimization: We generate preference pairs by applying graph edits to ground-truth LDs, then train the model to prefer correct LD using Direct Preference Optimization (DPO).

Proposed Method

Graph editing

Algorithm 1 Graph Editing

Input: Random seed set S with $|S| = s$, Deletion ratio τ, Graph $G = (V, E)$ with $|V(G)| = n$ nodes and $|E(G)| = m$ edges
// Initialize the graph
$G' \leftarrow G, \mathcal{G} \leftarrow \emptyset$;
// Compute number of nodes to delete
$k \leftarrow \lfloor \tau \cdot |V(G')| \rfloor$;
for each i in S **do**
  if $k > 0$ **then**
    // Randomly select and reconnect nodes
    $\mathcal{X} \leftarrow$ random k nodes from G';
    Let $\mathcal{W} \rightarrow \mathcal{X} \rightarrow \mathcal{Y}$ be a path in G';
    Connect $\mathcal{W} \rightarrow \mathcal{Y}$, preserving the connectivity;
    // Remove selected nodes
    Remove node $\mathcal{X}$ from G';
  else
    // If no node is removed, copy a random node
    Copy a random node-edge pair $(v_{\text{copy}}, e_{\text{copy}}) \in G'$;
    Connect a randomly selected node v_{orig} in G' to v_{copy} using e_{copy};
  end if
  // Store the modified graph
  Add G' to set $\mathcal{G}$;
end for
Return $\mathcal{G} = \{G'_1, G'_2, \ldots, G'_s\}$

- A graph editing algorithm that reflects the structural characteristics of node-edge compositions.
- Node and edge pairs are removed based on a predefined deletion ratio.
- For graphs that are too small to satisfy the deletion criterion, nodes and edges are instead duplicated and appended to the original graph.

Proposed Method

Graph editing

$$\mathrm{GED}(G, G') = \min_{T \in \mathcal{T}(G,G')} \sum_{e \in T} \mathrm{cost}(e)$$

- A graph editing algorithm that reflects the structural characteristics of node-edge compositions.
- Node and edge pairs are removed based on a predefined deletion ratio.
- For graphs that are too small to satisfy the deletion criterion, nodes and edges are instead duplicated and appended to the original graph.

Experimental Result

Main results

- Model: Llama3.1-8B-Instruct / Qwen2.5-7B-Instruct

Format	Method	Node F1	Edge F1	Node EM	Edge EM	Program EM
XML	SFT	84.8 / 79.0	74.7 / 68.0	55.2 / 46.4	54.8 / 45.6	54.8 / 45.6
	RAFT-V	88.1 / **87.4**	80.9 / 78.7	63.8 / 59.6	62.8 / 58.8	62.8 / 58.8
	Ours	**89.6 / 87.4**	**82.6 / 79.0**	**66.2 / 61.2**	**65.2 / 60.0**	**65.2 / 60.0**
JSON	SFT	85.1 / 81.2	74.9 / 69.3	53.6 / 46.0	52.6 / 45.4	52.6 / 45.4
	RAFT-V	90.1 / 88.4	82.0 / 79.6	63.8 / 59.2	63.2 / 57.4	63.2 / 57.4
	Ours	**90.7 / 88.7**	**83.0 / 80.5**	**66.0 / 61.6**	**65.2 / 60.2**	**65.2 / 60.2**
Metaprogram	SFT	86.3 / 81.7	75.7 / 70.2	55.2 / 47.6	54.0 / 47.2	54.0 / 47.2
	RAFT-V	89.9 / 89.1	82.4 / 80.4	64.6 / 60.6	63.8 / 59.6	63.8 / 59.6
	Ours	**90.6 / 89.7**	**83.6 / 81.6**	**68.6 / 63.2**	**67.2 / 61.8**	**67.2 / 61.8**

Experimental Result

Graph editing vs. Best-of-N sampling

BoN (Random)	90.4	83.2	66.8	65.6	65.6
BoN (Unstrict)	**90.7**	83.5	67.0	65.8	65.8
BoN (Strict)	90.5	83.3	67.4	66.2	66.2
Ours (Random)	90.6	83.5	68.0	66.6	66.6
Ours	90.6	**83.6**	**68.6**	**67.2**	**67.2**

- BoN sampling often fails to generate challenging cases consistently, as negative samples are selected from the sampled outputs.

- Our editing-based negative selection provides a more systematic approach to generating negative samples.

Experimental Result

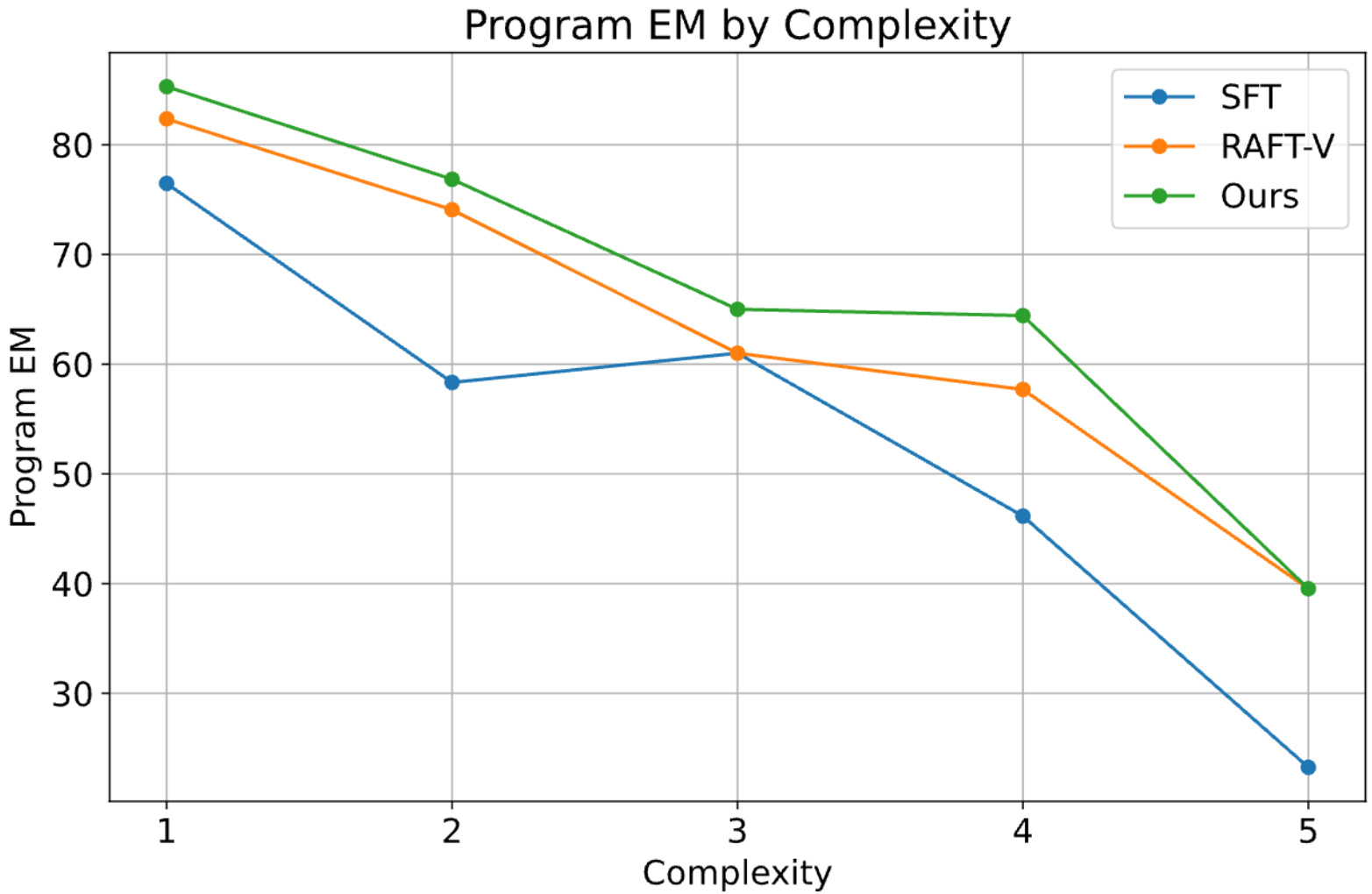
Deletion ratio in graph-editing algorithm

τ	Node F1	Edge F1	Node EM	Edge EM	Program EM
0	**90.7**	83.4	67.2	65.8	65.8
0.1 (Ours)	90.6	**83.6**	**68.6**	**67.2**	**67.2**
0.5	90.5	83.3	68.2	66.8	66.8
0.9	90.6	83.4	67.4	66.0	66.0

- Exact match (EM) scores decrease as τ increases
- When τ is 0.9, the model can easily distinguish them, making the training less effective.

Experimental Result

Performance across program complexity



Conclusion

We introduce: 2-stage training-based VPL generation methods.

- Demonstrate that training-based methods outperform prompting for industrial VPL (LD) generation.
- Propose a 2-stage approach with retrieval and preference optimization, achieving notable gains over SFT.
- Present the first LLM-based LD generator deployable in real-world factory settings.

E-mali: {deokhk, jeonghuncho} (at) postech.ac.kr

Paper